



Database Localization Design

Implementation Report

Sprint 6 • April 2026

Implementation status

This report reflects the implemented source code as of Sprint 6 updates: Java resource-bundle UI localization, user language persistence, multilingual flashcard definition storage, multilingual CSV import, and English fallback behavior.

1. Executive Summary

The original database localization plan proposed a generic translation-table design. During implementation, the team chose a smaller and more direct design that fits the current JavaFX application and sprint scope. Static interface text is localized with Java resource bundles, user language preference is stored in the USER table, and flashcard definitions are stored in language-specific columns on the FLASHCARD table.

The implemented design supports English, Arabic, Finnish, Korean, Lao, and Vietnamese. Students see flashcard definitions in their saved language where available, and the system falls back to English when a localized definition is missing.

2. Scope of Implementation

- Localize static JavaFX UI text through Messages resource bundles.
- Persist each user's selected language in USER.Language.
- Load the saved language after login and reload screens when the user changes language.
- Support right-to-left orientation for Arabic and Lao font handling for reliable glyph rendering.
- Store multilingual flashcard definitions for all supported languages.
- Import multilingual flashcard definitions from teacher-uploaded CSV, TSV, PSV, or pipe files.
- Display a student's flashcard definition in the student's preferred language, with English as fallback.
- Allow teachers to edit English, Arabic, Finnish, Korean, Lao, and Vietnamese definitions in the flashcard set detail screen.

3. Supported Languages

Language	Code	Locale	Primary Use
English	en	en_US	Default UI and flashcard definition fallback
Arabic	ar	ar_AR	UI localization, RTL support, flashcard definitions
Finnish	fi	fi_FI	UI localization and flashcard definitions
Korean	ko	ko_KR	UI localization and flashcard definitions
Lao	lo	lo_LA	UI localization, font support, flashcard definitions
Vietnamese	vi	vi_VN	UI localization and flashcard definitions

4. Static UI Localization

Static text is not stored in database translation tables. Instead, the app uses Java ResourceBundle files in src/main/resources. FXML files reference localized keys with the percent syntax, and Java controllers use ResourceBundle, I18n, or LocalizationService for programmatic text.

Resource File	Purpose
Messages.properties	Default fallback bundle
Messages_en_US.properties	English UI text
Messages_ar_AR.properties	Arabic UI text

Messages_fi_FI.properties	Finnish UI text
Messages_ko_KR.properties	Korean UI text
Messages_lo_LA.properties	Lao UI text
Messages_vi_VN.properties	Vietnamese UI text

```
<MenuButton text="%language.current"/>
<Button text="%account.logout"/>
```

5. Language Selection Flow

The user's language preference is represented by a two-letter code and stored directly in the USER table. LocaleManager resolves those codes to supported Java Locale values. When a user changes language from the Account screen, the user record is updated and the current screen is reloaded so JavaFX can resolve text with the selected bundle.

1. User opens Account and selects a language from the language menu.
2. AccountController calls LocaleManager.setLocaleByLanguage(language).
3. The current User object is updated with the selected language code.
4. UserService.update(user) persists USER.Language.
5. Navigator.reloadCurrent() reloads the active FXML with the selected ResourceBundle.
6. After login, LoginController applies the stored language before navigating into the app.

Database Field	Type	Description
USER.Language	VARCHAR(2)	Stores en, ar, fi, ko, lo, or vi. English is used when the value is missing or unsupported.

6. Flashcard Definition Localization

Dynamic flashcard content is localized only for definitions. Terms remain shared across languages so that students can learn the same technical term while seeing the explanation in their preferred language. The implemented schema stores each supported language in a dedicated column on FLASHCARD.

Column	Language	Required	Runtime Role
Definition	English	Yes	Fallback definition used when the selected language has no value
DefinitionAr	Arabic	No	Displayed for users with language ar
DefinitionFi	Finnish	No	Displayed for users with language fi
DefinitionKo	Korean	No	Displayed for users with language ko
DefinitionLo	Lao	No	Displayed for users with language lo
DefinitionVi	Vietnamese	No	Displayed for users with language vi

```

public String getLocalizedDefinition(String language) {
    String selected = switch (normalizeLanguage(language)) {
        case "ar" -> definitionAr;
        case "fi" -> definitionFi;
        case "ko" -> definitionKo;
        case "lo" -> definitionLo;
        case "vi" -> definitionVi;
        default -> definition;
    };
    return isBlank(selected) ? definition : selected;
}

```

7. Teacher Upload Format

Teachers can upload flashcard sets with multilingual definitions. FlashcardFileParser accepts the existing two-column format and the new multilingual format. The parser maps language columns by common language names or codes.

Format	Header Example	Behavior
Legacy	Term,Definition	Stores the second column as the English fallback definition.
Localized	Term,English,Arabic,Finnish,Korean,Lao,Vietnamese	Stores each language-specific definition in the matching FLASHCARD column.

CPU, Central Processing Unit, وحدة المعالجة المركزية, Keskusyksikkö, 중앙 처리 장치, ໜ່ວຍປະມວນຜົນກາງ, Bộ xử lý trung tâm

8. Teacher Editing Workflow

The teacher flashcard set detail screen now exposes separate text areas for all supported definition languages. English is required because it is the fallback. Arabic, Finnish, Korean, Lao, and Vietnamese are optional and may be added later.

- Add mode clears all six definition inputs.
- Edit mode loads existing Definition, DefinitionAr, DefinitionFi, DefinitionKo, DefinitionLo, and DefinitionVi values.
- Save writes all language fields back to the Flashcard entity.
- The list preview continues to show the English fallback definition so the teacher has a stable default view.

9. Student Runtime Behavior

When a student views flashcard details, FlashcardDetailController reads the current user's language and calls getLocalizedDefinition. This keeps the UI simple while allowing the learning content to adapt to the learner's preferred language.

Scenario	Displayed Definition
Student language is vi and DefinitionVi has text	Vietnamese definition
Student language is lo and DefinitionLo has text	Lao definition
Student language is ko but DefinitionKo is blank	English Definition fallback
User language is missing	English Definition fallback

10. Differences From the Initial Database Plan

Initial Plan	Implemented Design	Reason
Dedicated Language table	Supported languages are defined in LocaleManager and resource files	Simpler for fixed six-language scope
LocalizedString database table	Java properties resource bundles	Matches JavaFX ResourceBundle and FXML key support
Generic translation tables for dynamic content	Dedicated FLASHCARD definition columns	Lower implementation risk and easier JPA mapping for the current feature
Excel import service for UI text	CSV/TSV/pipe parser for flashcard content only	Teacher upload requirement focuses on learning content rather than UI labels

11. Constraints and Risks

- The language list is fixed in code. Adding a seventh language requires adding a resource bundle, LocaleManager mapping, CSV parser mapping, Flashcard entity field, and database column.
- Only flashcard definitions are localized as dynamic content. Terms, class names, and set names are still shared text.
- The current column-based schema is simple but less flexible than a generic translation table if the product later needs many languages.
- Resource bundle files must be saved as UTF-8 without BOM. A BOM before the first key prevents Java from resolving that key correctly.
- Teacher-entered localized definitions should be reviewed for quality because the application stores provided text and does not translate automatically.

12. Future Improvements

- Move to a FlashcardDefinitionTranslation table if the supported language list grows beyond the current fixed set.
- Add validation feedback when an uploaded file is missing the English fallback definition.
- Persist teacher inline edits immediately through FlashcardService.update for stronger consistency.

- Add UI indicators showing which language definitions are missing for each card.
- Consider localizing quiz answer definitions with the same fallback strategy used by flashcard details.

13. Conclusion

The implemented localization design is intentionally pragmatic. It provides multilingual UI support and localized flashcard definitions without adding complex translation-table infrastructure. This fits the current Fliply scope: six known languages, JavaFX screens backed by ResourceBundle files, teacher-uploaded learning content, and student-facing definition fallback to English.